



cpi'fp

Los Enlaces

DESARROLLO WEB EN ENTORNO SERVIDOR

2º DESARROLLO DE APLICACIONES WEB

BBDD EN LARAVEL

1. MIGRACIONES

Las **migraciones** constituyen una especie de control de versiones para la base de datos de la aplicación. Permiten crear y modificar tablas de la BD con independencia del SGBD que estemos usando.

Con las migraciones no solo podrás reconstruir la base de datos, sino que podrás parchear la BD de una aplicación en producción en un tiempo récord y con riesgo cero.



1. MIGRACIONES

Antes de empezar, recuerda que, para que cualquier operación sobre la base de datos funcione, debes tener bien configuradas estas variables de entorno del archivo `.env` de Laravel:

```
DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=mi-base-de-datos
DB_USERNAME=mi-usuario-de-BD
DB_PASSWORD=mi-password-de-BD
```



1. MIGRACIONES

La primera cosa para la que sirven las migraciones es para crear las tablas de tu aplicación. Olvídate de exportar e importar a mano el archivo SQL de tu base de datos. Un ejemplo: imagina que tenemos una tabla llamada *Clients* con los campos típicos (*id*, *name*, *address*, etc...).

Paso 1. Inicializar el sistema de migraciones de Laravel (si ya lo hemos hecho antes, nos dará un error al intentar hacerlo otra vez):

```
$ php artisan migrate:install
```

1. MIGRACIONES

Paso 2. Crear la migración para la tabla Clients:

```
$ php artisan make:migration create_clients_table
```

Esto generará un fichero en la carpeta `/database/migrations` cuyo nombre contendrá un *timestamp* o marca de tiempo. Si editas ese fichero, verás dos métodos:

- **up()** → se ejecuta cuando se lanza la migración. Se encarga de crear la tabla en la base de datos.
- **down()** → se ejecuta cuando se cancela la migración. Se encarga de eliminar la tabla de la base de datos.



1. MIGRACIONES

Paso 3. Editar el fichero `/database/migrations/20221226072434createclientstable.php`. En el método `up()` tienes que indicar las columnas que tendrá la tabla:

```
public function up() {  
    Schema::create('clients', function (Blueprint $table) {  
        $table->bigIncrements('id')->index(); // UNSIGNED BIGINT AUTOINC.  
        $table->string('name', 75)->unique(); // VARCHAR  
        $table->text('address')->nullable(); // TEXT  
        $table->integer('level'); // INT  
        $table->date('birth_date'); // DATE  
        $table->timestamps(); // Crea los campos created_at y updated_at  
    });  
}
```

1. MIGRACIONES

Paso 4. Lanzar las migraciones:

```
$ php artisan migrate
```

Paso 5. Revertir las migraciones (si es necesario)

Si necesitas revertir la creación de todas las tablas:

```
$ php artisan migrate:rollback
```

Para revertir solo el último paso en la creación de tablas:

```
$ php artisan migrate:rollback --step=1
```



1. MIGRACIONES

Para dejar la BD a su estado original (vacía):

```
$ php artisan migrate:reset
```

¡Cuidado! Estas acciones son **destructivas**. Pero, por supuesto, hay una forma de modificar una tabla sin borrarla y volver a crearla. Es lo que vamos a hacer a continuación.

2. MODIFICAR TABLAS MEDIANTE MIGRACIONES

Si necesitas **modificar una tabla** que ya existe (por ejemplo, para añadir o eliminar campos), tienes dos opciones:

1. **Modificar la migración original** (en la que se crea la tabla) para añadir o eliminar el campo afectado. Esto te obligará a lanzar la migración de nuevo y, por lo tanto, la tabla se reconstruirá y todos los datos que pudiera contener se perderán.
2. **Crear una nueva migración** en la que únicamente se haga la modificación de la tabla, sin tocar el resto. Esto respetará los datos que la tabla ya pudiera contener.

2. MODIFICAR TABLAS MEDIANTE MIGRACIONES

Como es lógico, la opción 2 será la que preferiremos si la aplicación ya está en producción y necesitamos modificar la estructura de la base de datos. En cambio, durante el desarrollo, puede ser más simple utilizar la opción 1.

Supongamos que queremos añadir un campo *email* a la tabla *Clients* del apartado anterior. Si optas por la opción 2, es decir, por crear una nueva migración que se encargue de hacer esa modificación en la tabla sin alterar sus datos, la forma de proceder es la siguiente:

2. MODIFICAR TABLAS MEDIANTE MIGRACIONES

Paso 1. Crear la migración:

```
$ php artisan make:migration add_email_to_clients --table=clients
```

(Nota: puedes asignarles el nombre que quieras, pero Laravel aconseja utilizar las convenciones que ves en estos ejemplos)

Paso 2. Editar la migración `/database/migration/add_email_to_clients.php` para añadir, en el método `up()`, el campo nuevo; y, en el método `down()`, especificaremos qué hay que hacer en caso de que se fuerce un rollback de esta migración:



2. MODIFICAR TABLAS MEDIANTE MIGRACIONES

```
public function up() {  
    Schema::table('clients', function (Blueprint $table) {  
        $table->string('email')->after('address');  
    });  
}  
  
public function down() {  
    Schema::table('clients', function (Blueprint $table) {  
        $table->dropColumn('email');  
    });  
}
```



2. MODIFICAR TABLAS MEDIANTE MIGRACIONES

Las migraciones pueden usarse para cualquier otra operación sobre la estructura de la base de datos, como:

- Cambiar tipos de columnas.
- Cambiar atributos de columnas (null, unique, default...)
- Cambiar o asignar claves primarias y ajenas.

Más info en: <https://laravel.com/docs/10.x/migrations>

3. SEEDING

El **seeding** es una técnica adicional a la de las migraciones que permite **cargar con datos las tablas** de la base de datos. Es muy práctico:

- Si quieres tener un conjunto de **datos de prueba** en tu base de datos de desarrollo, esa que destrozas periódicamente cuando haces pruebas.
- Si necesitas **cargar algunos datos mínimos** en algunas tablas para que la aplicación funcione una vez desplegada en un servidor de producción.

3. SEEDING

Para crear un *seeder* (p.ej, para la tabla *users*), sigue estos pasos:

Paso 1. Ejecuta el comando

```
$ php artisan make:seeder UsersTableSeeder
```

Paso 2. Edita el archivo `/database/seeds/UsersTableSeeder.php` y añade algo como esto al método `up()` (por supuesto, modifica el código para adaptarlo a tu tabla y a tus datos):



3. SEEDING

```
use Illuminate\Support\Facades\DB;

public function run() {
    Users::truncate(); // Optativo: vacía la tabla antes de rellenarla
    DB::table('users')->insert([
        'name' => 'Stephen Falken',
        'address' => 'Oregon 97, Goose Island',
        'email' => 'sfalken@norad.com',
        'birth_date' => '1932-09-03',
    ]);
    DB::table('users')->insert([...]);
    ...
}
```




3. SEEDING

Paso 3. Ejecuta este comando para lanzar el seeder y que los datos se carguen en tu tabla:

```
$ php artisan db:seed --class=UsersTableSeeder
```

Esto cargará tantos registros en la tabla *users* como líneas *insert()* hayas incluido en el método *up()*.

3. SEEDING

Automatizar el seeding:

Paso 1. Edita el fichero `/database/seeds/DatabaseSeeder.php`

Paso 2. Añade a la función `run()` una línea como esta por cada seeder que quieras ejecutar automáticamente:

```
$this->call(UsersTableSeeder::class);
```

Paso 3. ¡Y listo! Al ejecutar el comando `db:seed` de Artisan, sin indicar la clase, se lanzarán todos los seeders que hayas añadido a `run()`.

```
$ php artisan db:seed
```

4. COMANDOS PARA MIGRACIONES

- Lanzar todas las migraciones pendientes:

```
$ php artisan migrate
```

- Crear la migración para crear una tabla:

```
$ php artisan make:migration <nombre> --create=<tabla>
```

- Crear una migración para modificar una tabla ya existente:

```
$ php artisan make:migration <nombre> --table=<tabla>
```

- Retroceder un paso en todas las migraciones:

```
$ php artisan migrate:rollback
```

4. COMANDOS PARA MIGRACIONES

- Retroceder N pasos en todas las migraciones:

```
$ php artisan migrate:rollback --step=<N>
```

- Deshacer todas las migraciones (resetear la BD):

```
$ php artisan migrate:reset
```

- Resetear la BD y reconstruirla lanzando todas las migraciones:

```
$ php artisan migrate:refresh
```

- Resetear la BD, reconstruirla y llenarla de datos con los seeders:

```
$ php artisan migrate:refresh --seed
```

4. COMANDOS PARA MIGRACIONES

- Eliminar todas las tablas y lanzar todas las migraciones de nuevo:

```
$ php artisan migrate:fresh
```

- Eliminar todas las tablas y lanzar todas las migraciones y seeders:

```
$ php artisan migrate:fresh --seed
```

Diferencias entre los comandos `migrate:refresh` y `migrate:fresh`:

<https://codea.app/blog/refresh-vs-fesh>



5. ELOQUENT

Eloquent no es más que una librería incluida con Laravel que utiliza un patrón de software llamado **ORM (Object-Relational Mapping)** para abstraer aún más el acceso a la base de datos, de manera que no tengamos que escribir y depurar SQL.

Mapear los objetos de nuestra aplicación con una BD relacional significa que Eloquent convierte los registros de tus tablas en objetos de tu aplicación para que los manipules con mayor facilidad.

5. ELOQUENT

Podrás manejar los datos de tu base de datos como si fueran objetos de tu aplicación. Y, cuando los modifiques, borres o crees, se ejecutará el código SQL necesario para traducir esas operaciones en sentencias para la base de datos.

Imagina que tenemos una tabla de artículos de un blog. La llamaremos *Articles*, y tendrá 3 campos: el *id* (entero), el *title* (cadena de caracteres) y el *body* (cadena de caracteres). Con Eloquent, usar esa tabla es tan fácil como hacer algo así:



5. ELOQUENT

```
// Buscamos un artículo por su id
$art = Article::find('7');

// Ahora podemos acceder a los campos de ese artículo
echo $art->title;

// O también podemos modificar los campos del artículo
$art->body = "Texto del cuerpo";

// Si hacemos save(), los cambios se guardan en la BD
$art->save();
```




5. ELOQUENT

Para usar Eloquent en tu aplicación tienes que crear un modelo:

```
$ php artisan make:model Article
```

El modelo se creará en */app/models/Article.php*. Si creas el modelo con la opción *-m*, se creará automáticamente su migración, lo cual resulta tremendamente práctico:

```
$ php artisan make:model Article -m
```

5. ELOQUENT

Laravel supondrá que **la tabla se llama igual que el modelo**, solo que en **minúscula y plural**. Es decir, la tabla de la base de datos asociada al modelo *Article* se debería llamar *articles*.

Recuerda que en Laravel existen un montón de convenciones sobre los nombres de las cosas que conviene respetar, más que nada porque te facilitan mucho la vida y hacen que te centres en lo verdaderamente importante (crear el código de tu aplicación) en lugar de en lo accesorio (pelearte con los nombres).

5. ELOQUENT

No obstante, si quieres ponerle otro nombre a la tabla, puedes hacerlo. También puedes cambiar otras muchas cosas, como el nombre del campo clave (Laravel supondrá que se llama id) o los campos sobre los que se puede hacer una asignación masiva, algo muy útil que veremos un poco más adelante.

Por ejemplo, podemos **editar el modelo** `/app/Articles.php` y añadir estas líneas dentro de nuestra clase:



5. ELOQUENT

```
// El nombre de la tabla no será "articles" sino "articulos"
protected $table = 'articulos';

// La clave primaria no será "id" sino "id_art"
protected $primaryKey = 'id_art';

// Campos de la tabla en los que se permite la ASIGNACIÓN MASIVA (más adelante
veremos qué es esto)
protected $fillable = array('id','titulo','cuerpo');
```



5. ELOQUENT

Solo con construir el modelo (y asignarlo a la tabla adecuada) ya tenemos detrás a Eloquent haciendo el mapeo objeto-relacional.

Ahora podemos ir a nuestro controlador (que, por respetar las convenciones de Laravel, debería llamarse *ArticlesController*) y **lanzar consultas (SELECT)** con expresiones como las que se muestran en la siguiente diapositiva:



5. ELOQUENT

```
$articlesList = Article::all();           // Devuelve todos los artículos
// Devuelve el artículo con ese $id
$myArticle = Article::find($id);
// Error 404 si el artículo no existe
$myArticle = Article::findOrFail($id);
// SELECT con WHERE
$articlesList = Article::where('id', '>', 100)->get();
// SELECT con WHERE y TAKE
$articlesList = Article::where('id', '>', 100)->take(10)->get();
// Devuelve el último id asignado
$maxId = Article::max('id');
```



5. ELOQUENT

También podemos usar Eloquent para **insertar (INSERT)** un nuevo artículo desde nuestro controlador:

```
$art = new Article;  
$art->title = 'Los Chitauri invaden Nueva York';  
$art->body = 'Bla, bla, bla';  
$art->save();
```

5. ELOQUENT

Si los datos del artículo vienen de un formulario, se puede recoger esos datos, crear un objeto Article con ellos y guardar el artículo en la BD:

```
public function store(Request $request) {  
    // Esto es una ASIGNACIÓN MASIVA de las que hablábamos más arriba  
    Article::create($request->all());  
    return view('la-vista-que-sea');  
}
```

Ojo: solo los campos que hayas indicado como *fillable* en el modelo se podrán asignar al artículo de este modo. Eso es lo que significa **asignación masiva**.

5. ELOQUENT

Y, por supuesto, también podemos **modificar (UPDATE)** y **borrar (DELETE)** artículos de la base de datos:

```
// MODIFICAR
$art = Article::find(18);
$art->body = 'Nuevo cuerpo';
$art->save();
```

```
// BORRAR
$art = Article::find(13);
$art->delete();
```

5. ELOQUENT

Hemos visto en los últimos ejemplos algunos **métodos de Eloquent** por separado. Te los reúno en esta sección para que los puedas consultar.

(Aviso: no están todos, solo los de uso más habitual. Si quieres una lista completa, ya sabes: acude a la [documentación oficial](#))

- **all()** → Recupera todos los registros de una tabla.
- **where("campo", valor)** → Aplica cláusula WHERE.
- **orderBy("campo", "asc|desc")** → Aplica cláusula ORDER BY.
- **get()** → Recupera registros seleccionados. Se suele usar con *where* y/o *orderBy*.

5. ELOQUENT

- **first()** → Recupera el primer registro.
- **latest()** → Recupera el último registro.
- **find(valor)** → Busca registros con ese valor en el campo *id*.
- **findOrFail(valor)** → Lanza un error 404 si no encuentra el registro.
- **count(), max(), min()...** → Utiliza funciones de agregado de SQL.
- **save()** → Inserta o actualiza registros.
- **update()** → Actualiza registros.
- **delete()** → Elimina registros.

5. RELACIONES ENTRE TABLAS CON ELOQUENT

Las **relaciones entre tablas** también se pueden manejar con Eloquent sin necesidad de andar escribiendo larguísimos INNER JOIN y otros miembros de su nutrida familia, con todos los errores de escritura que suelen hacernos perder el tiempo depurando SQL.

Aunque al principio te parezca que definir las relaciones entre tablas con Eloquent necesita mucho **trabajo previo**, te garantizo que después te alegrarás de haberlo hecho. Porque las relaciones, una vez definidas, se comportan como consultas y se puede operar con ellas como si lo fueran.

5. RELACIONES ENTRE TABLAS CON ELOQUENT

Lo comprenderemos mejor, como siempre, con ejemplos. En los siguientes apartados, vamos a suponer que tenemos estas tablas:

- **usuarios**(id#, nombre, passwd)
- **emails**(id#, email, usuario_id) → Esta tabla tiene una relación 1:1 con usuarios
- **articulos**(id#, titulo, texto, **idUsuario**) → Esta tabla tiene una relación 1:N con usuarios (**nombre no estándar para la clave foránea**)
- **roles**(id#, nombre) → Esta tabla tiene una relación N:N con usuarios

5. RELACIONES ENTRE TABLAS CON ELOQUENT

RELACIONES 1:1 (USUARIOS $\longleftrightarrow$ EMAILS)

Paso 1. En el modelo de la tabla maestra (clase *Usuario*, en nuestro ejemplo) añadimos este método:

```
// Este usuario tiene un elemento email de la clase Email  
public function email() { return $this->hasOne('App\Models>Email'); }
```

Paso 2. En el modelo de la tabla relacionada (clase *Email*) añadimos:

```
// Este email pertenece a un usuario de la clase Usuario  
public function usuario() { return $this->belongsTo('App\Models\Usuario'); }
```

5. RELACIONES ENTRE TABLAS CON ELOQUENT

RELACIONES 1:1 (USUARIOS $\longleftrightarrow$ EMAILS)

A partir de ahora se puede recuperar el email de un usuario como si fuera un miembro de la clase *Usuario*, con algo tan sencillo como esto:

```
$email = Usuario::find(1)->email;  
$user = Email::all()->first()->user;
```

Y también funciona al revés. Es decir, a partir de un objeto de tipo *Email*, podemos acceder a su usuario como si fuera un atributo de la clase *Email*.

5. RELACIONES ENTRE TABLAS CON ELOQUENT

RELACIONES 1:N (USUARIOS $\longleftrightarrow$ ARTÍCULOS)

En una relación 1:N (como la que hay entre las tablas *usuarios* y *artículos* de nuestro ejemplo), para definirla en Eloquent tienes que hacer esto:

Paso 1. En el modelo de la tabla maestra (*Usuario*), añade este método:

```
public function articulos() {  
    return $this->hasMany('App\Models\Articulo', 'idUsuario');  
} // ATENCIÓN: hemos tenido que indicar el nombre de la clave foránea  
// (idUsuario) porque no habíamos respetado la convención de Laravel  
// (usuario_id) al crear la tabla de artículos
```


5. RELACIONES ENTRE TABLAS CON ELOQUENT

RELACIONES 1:N (USUARIOS $\longleftrightarrow$ ARTÍCULOS)

Paso 2. En el modelo de la tabla relacionada (class *Articulo*), añade este otro método:

```
public function usuario() { return $this->belongsTo('App\Models\Usuario'); }
```

¡Y listo! Ya puedes recuperar los artículos a partir del usuario, como si fueran atributos de esa clase. Y a la inversa también funciona.

```
$articulos = Usuario::find(1)->articulos;  
foreach ($articulos as $articulo) {  
    // Procesar cada artículo  
}
```

5. RELACIONES ENTRE TABLAS CON ELOQUENT

RELACIONES N:N (USUARIOS $\longleftrightarrow$ ROLES)

Si lo que tienes es una relación N:N (como la que hay entre *usuarios* y *roles* en nuestro ejemplo), los pasos a seguir para construirla con Eloquent son estos:

Paso 1. En el modelo de una de las tablas (clase *Usuario*) añadimos este método:

```
public function roles() {  
    return $this->belongsToMany('App\Models\Rol');  
}
```

5. RELACIONES ENTRE TABLAS CON ELOQUENT

RELACIONES N:N (USUARIOS $\longleftrightarrow$ ROLES)

Paso 2. En el modelo de la otra tabla (clase *Rol*) añadimos este método:

```
public function usuarios() {  
    return $this->belongsToMany('App\Models\Usuario'); }  

```

Ahora, ya se pueden recuperar los roles a partir del usuario o a la inversa. Por ejemplo:

```
$roles = Usuario::find(1)->roles;  
foreach ($roles as $rol) {  
    // Procesar cada rol }  

```

5. RELACIONES ENTRE TABLAS CON ELOQUENT

Insertar, modificar y borrar en relaciones N:N implica escribir datos (normalmente *ids*) en la **tabla intermedia** o **tabla pivote**, lo cual suele suponer un engorro cuando se hace *a mano* con SQL.

Para insertar un usuario y sus roles se usa el método *attach()*:

```
public function store(Request $r) {  
    $user = new User($r->all());  
    $user->roles()->attach($r->roles);  
    $user->save();  
}
```



5. RELACIONES ENTRE TABLAS CON ELOQUENT

Para actualizar un usuario y sus roles se usa el método *sync()*:

```
public function update(Request $r, $id) {  
    $user = User::find($id);  
    $user->fill($r->all());  
    $user->roles()->sync($r->roles);  
    $user->save();  
}
```



5. RELACIONES ENTRE TABLAS CON ELOQUENT

Para eliminar un usuario y sus roles se usa el método *detach()*:

```
public function destroy($id) {  
    $user = User::find($id);  
    $user->roles()->detach();  
    $user->delete();  
}
```

5. RELACIONES ENTRE TABLAS CON ELOQUENT

En relaciones N:N, Eloquent supondrá que el **nombre de la tabla** de la relación se ha formado con los nombres de las dos tablas maestras en snake case y ordenadas alfabéticamente. Por ejemplo, entre *usuarios* y *roles*, Eloquent supondrá que existe una tabla llamada *roles_usuarios*. Si no se llama así, la relación fallará. Se puede indicar otro nombre al definir la relación. Por ejemplo, en el modelo de usuarios (clase *Usuario*):

```
public function roles() {  
    return $this->belongsToMany('App\Models\Rol', 'usuarios_roles');  
}
```



5. RELACIONES ENTRE TABLAS CON ELOQUENT

También se pueden indicar los nombres de las claves foráneas si no siguen las convenciones (que, según Laravel, son *usuario_id*, *rol_id*, etc):

```
public function roles() {  
    return $this->belongsToMany('App\Models\Rol',  
                                'usuarios_rols',  
                                'id_usuario',  
                                'id_rol');  
}
```




5. RELACIONES ENTRE TABLAS CON ELOQUENT

Hemos creado un método para acceder a la tabla relacionada, pero estamos usando un atributo en su lugar:

```
public function articulos() { return $this->hasMany('App\Models\Articulo'); }  
public function loQueSea() { $arts = Usuario::find(1)->articulos; }  
// articulos, no articulos()
```

El atributo *articulo* es un atributo virtual creado por Eloquent. Pero el método *articulos()* también existe, y puede usarse como una consulta, extendiéndola como necesitemos. Por ejemplo:

```
$arts = Usuario::find(1)->articulos()->where('titulo','foo')->first();
```

6. QUERY BUILDER

Eloquent permite usar la BD de forma simple y elegante en la mayor parte de las circunstancias. Aún así, puede haber situaciones en las que queramos un acceso de más bajo nivel a la BD. Para eso existe **QueryBuilder**.

El grado de abstracción de QueryBuilder es mucho menor que el de Eloquent. Es decir, estaremos *CASI escribiendo SQL*, sin llegar a hacerlo. No tendrás que depurar el SQL, ni pelearte con comillas que se abren y cierran, ni nada de eso, QueryBuilder generará el SQL por ti.



6. QUERY BUILDER

Algunos ejemplos de uso:

```
$users = DB::table("users")->get();  
$users = DB::table("users")->where("name", "=", "Ana")->first();  
$users = DB::table("users")->where("edad", ">=", 18)->orderBy("apellidos");  
$maxId = DB::table("users")->max("id");  
$existe = DB::table("users")->where("id", "=", $id)->exists();  
$users = DB::table("users")->select("nombre, apellidos as apell")->get();
```

En la [documentación oficial](#) encontrarás una referencia completa de todas las funciones de QueryBuilder, pero con estas que ves en el ejemplo puedes construir prácticamente cualquier consulta sencilla.

6. QUERY BUILDER

El **resultado** de consultas como las del ejemplo es bastante intuitivo:

- O bien un **dato simple** (como el *\$maxId* de la cuarta consulta, que es un entero).
- O bien un **objeto de tipo Collection**.

Las colecciones de Laravel tienen un montón de métodos útiles para procesarlas, pero la mayor parte de las veces basta con hacer un *foreach* sobre la variable para ir accediendo a cada uno de los elementos, que se comportarán como objetos del tipo adecuado.



6. QUERY BUILDER

Por ejemplo, para acceder a todos los registros de la tabla de usuarios:

```
$users = DB::table("users")->get();  
foreach ($users as $user) {  
    echo $user->name;  
    echo $user->email;  
    ...etc...  
}
```

6. QUERY BUILDER

Ventajas de QueryBuilder sobre SQL:

1. No tendremos que depurar nuestros errores sintácticos en SQL, con el ahorro de tiempo que eso conlleva.
2. El SQL generado será 100% compatible con el gestor de base de datos que estemos utilizando. Si escribimos SQL en crudo, tendremos que adaptarlo al dialecto de nuestro gestor de base de datos. Y, si cambiamos de gestor, habrá que revisar todas las sentencias SQL para adaptarlas de nuevo. Todo esto lo evita QueryBuilder, puesto que hace esa adaptación por nosotros.
3. Es imposible que suframos un ataque por inyección de código, puesto que QueryBuilder no lo permitirá.



7. RELACIONES ENTRE TABLAS CON QUERY BUILDER

Las relaciones entre tablas se manejan con *joins*, como en SQL, solo que escritos al estilo QueryBuilder. Para hacer un **INNER JOIN**, puedes usar como referencia este ejemplo:

```
$users = DB::table('users')
    ->join('contacts', 'users.id', '=', 'contacts.user_id')
    ->join('orders', 'users.id', '=', 'orders.user_id')
    ->select('users.*', 'contacts.phone', 'orders.price')
    ->get();
```

8. SQL CON QUERY BUILDER

Por último, QueryBuilder también te permite escribir SQL crudo, es decir, SQL tal cual, si es que alguna vez lo necesitas. Eso sí, deberías valorar muy bien para qué narices quieres escribir SQL crudo. ¿Estás seguro/a de que eso que intentas hacer no se puede lograr más fácilmente con Eloquent o con QueryBuilder? Además, tendrás que extremar las precauciones ante un posible ataque por inyección de código.

Si aún así no te he convencido, puedes ejecutar tu SQL crudo así:

```
$resultado = DB::raw('escribe-aquí-tu-sentencia-SQL');
```